

University of Agriculture Faisalabad
Department of Computer Science
Faculty of Sciences

MyDayX
Personal Productivity & Budget Manager
TECHNICAL REPORT

SUBMITTED BY

ARSLA AYESHA

2024-ag-6349

ADVISED BY

Dr. QAMAR NAWAZ

A TECHNICAL REPORT SUBMITTED IN PARTIAL FULFILLMENT OF
REQUIREMENT FOR THE DEGREE OF
BACHELOR OF SCIENCE IN COMPUTER SCIENCE

DECLARATION

I hereby declare that the contents of the report "MyDayX – Personal Productivity & Budget Manager" is a project of my own and no part has been copied from any published source (except the references). I further declare that this work has not been submitted for award of any other diploma/degree. The university may take action if the information provided is found false at any stage. In case of any default the scholar will be proceeded against as per UAF policy.

ARSLA AYESHA

2024-ag-6349

CERTIFICATE

To,
The Controller of Examinations,
University of Agriculture,
Faisalabad.

The supervisory committee certifies that the ARSLA AYESHA bearing the 2024-ag-6349 has successfully completed the project "MyDayX – Personal Productivity & Budget Manager" in partial fulfillment of requirement for the degree of B.Sc. Computer Science under our guidance and supervision.

Dr.QAMAR NAWAZ
Department of Computer Science

Dr. Muhammad Ahsan Latif
Chairman, Department of Computer Science

ACKNOWLEDGEMENT

All praise is for Almighty Allah who gave us the strength and ability to complete this project successfully. We thank everyone who contributed, in one way or another, to the completion of this technical report. We are deeply grateful to the Department of Computer Science, University of Agriculture Faisalabad, for providing us the opportunity to study in such a prestigious institution.

Our heartfelt gratitude goes to our supervisor for his/her continuous guidance, encouragement, and invaluable support throughout the entire development process. It is under his/her able supervision that this work came into its final shape. For any shortcomings, we take full responsibility.

We are also thankful to our fellow students for their constructive criticism and thought-provoking suggestions that helped us improve our work. Last but not least, we extend our deepest thanks to our families for their endless prayers, patience, and moral support. May Almighty Allah richly bless all of you.

ABSTRACT

MyDayX is a desktop-based personal productivity and budget management application developed using C# Windows Forms with a SQLite backend. The application is designed to help individuals effectively organize their daily tasks and track their financial activities in a single unified platform. In an era where time management and financial discipline are essential for personal success, MyDayX provides a lightweight yet feature-rich solution that does not require internet connectivity or complex server infrastructure.

The system provides secure user authentication through a registration and login module, followed by a personalized dashboard that displays a real-time summary of income, expenses, savings, and budget status. Users can manage daily tasks by adding them and marking them complete, track income and expenditure under categorized headings, set monthly budget limits, and monitor their financial performance through visual charts and analytics. The application further offers a detailed transaction history with filtering and pagination capabilities, as well as a performance module that compares category-wise spending against the allocated budget limit.

The development followed the Agile process model, with incremental delivery of functional modules. The technology stack consists of C# (Windows Forms), SQLite for local data storage, and the Windows Forms charting library for data visualization. The database schema comprises five tables: Users, Tasks, Income, Expenses, and Budget, all linked through foreign key relationships. The repository pattern was adopted to cleanly separate data access logic from the presentation layer.

Testing was conducted using black-box functional test cases covering all major use cases including login, sign-up, adding income/expenses, setting budget limits, viewing history, and generating reports. All test cases passed successfully, confirming that the system meets its specified functional and non-functional requirements. The user interface is clean, responsive, and intuitive, making it accessible to users with minimal technical background. MyDayX addresses a common gap in personal finance tools by combining both task and budget management into one offline desktop application.

Contents

CHAPTER 1 – INTRODUCTION	7
1.1 Background.....	7
1.2 Description.....	7
1.3 Problem Statement.....	7
1.4 Scope	8
1.5 Objectives	8
1.6 Feasibility	8
1.7 Requirements	9
1.7.1 Functional Requirements.....	9
1.7.2 Non-Functional Requirements	10
1.7.3 Hardware Requirements	10
1.7.4 Software Requirements	10
1.8 Stakeholders.....	11
2.1 Process Model.....	12
2.2 Tools & Technologies	12
2.3 Design.....	12
2.3.1 Use Case Diagrams	12
2.3.2 Usage Scenarios	13
2.3.3 Sequence Diagram	15
2.3.4 Class Diagram	16
2.3.5 Data Flow Diagram	16
2.3.6 ER Diagram.....	17
2.3.7 Database Model.....	17
2.3.8 Architecture.....	18
3.1 Testing	19
3.2 Test Cases.....	19
3.3 Conclusion	22
CHAPTER 4 – USER MANUAL.....	23
4.1 Login Screen.....	23
4.2 Sign Up Screen	23
4.3 Dashboard.....	24
4.4 Budget Module	24
4.5 History Module.....	25
4.6 Charts Module	25
4.7 Performance Module	26
REFERENCES	27

CHAPTER 1 – INTRODUCTION

1.1 Background

In the modern age, individuals face increasing challenges in managing their daily schedules and personal finances simultaneously. The lack of integrated tools that bring together both task management and budget tracking compels people to rely on multiple separate applications, leading to fragmented productivity. MyDayX was conceived to address this growing need by offering a unified desktop platform that allows users to plan their day and manage their money in a single application.

The concept of a personal productivity system is not new; however, most existing solutions are either cloud-dependent, subscription-based, or focused solely on one domain. MyDayX fills this gap by providing an offline-first desktop application built with C# and Windows Forms, backed by a local SQLite database. The idea originated from the everyday struggles of university students and working professionals who need a simple, cost-free, and privacy-respecting tool.

By combining task management with financial tracking, MyDayX empowers users to see the bigger picture of their day: what they need to accomplish and how much they are spending in pursuit of their goals. The application supports features such as income/expense categorization, daily/weekly/monthly reporting, visual charts, and budget limit warnings, all within an intuitive Windows interface. The project was developed at the Department of Computer Science, University of Agriculture Faisalabad, as a technical project.

1.2 Description

MyDayX – Personal Productivity & Budget Manager is a fully functional desktop application built on the .NET 8 (C# Windows Forms) framework with SQLite as its embedded database engine. The application targets individual users who wish to keep track of their daily tasks and financial transactions without relying on external internet services or paid subscriptions.

The application begins with a secure Login and Registration module that enforces basic validation rules such as unique usernames, valid email format, and minimum password length. Once authenticated, the user is directed to a multi-panel Dashboard that displays a real-time summary of their financial statistics, categorized by daily, weekly, or monthly periods.

The Budget Module allows users to add income entries (categorized under Salary, Freelance, Business, Investment, or Other) and expense entries (categorized under Food, Travel, Study, Shopping, Health, or Other). Users can also define a monthly budget limit by allocating amounts across three sub-categories (Food, Travel, and Study), which the application sums and monitors in real time, issuing a warning when 90% of the combined limit is reached and an alert when it is exceeded. The History Module presents a paginated, filterable view of all past transactions. The Charts Module provides bar and pie charts. The Performance Module breaks down spending per category against the budget limit.

1.3 Problem Statement

Individuals, particularly students and young professionals, struggle to maintain control over their daily productivity and personal finances due to the absence of a single integrated tool that covers both dimensions. Most available solutions are either internet-dependent, expensive, or limited to a single domain, requiring users to switch between multiple applications. This fragmentation results in poor time management, unmonitored overspending, and general loss of productivity. MyDayX addresses this problem by delivering an offline, free-to-use, integrated desktop application with real-time alerts and visual analytics.

1.4 Scope

MyDayX is scoped as a desktop application for individual personal use on the Windows platform. The system encompasses:

- User account management (registration, login, logout)
- Daily task creation and completion tracking
- Income and expense entry with category classification
- Monthly budget limit setting and monitoring
- Transaction history with filtering and pagination
- Visual charts for income vs. expense and category-wise breakdown
- Performance reporting with progress indicators

The application does not include multi-user collaboration, cloud synchronization, mobile platforms, or bank account integration within the current scope.

1.5 Objectives

The primary goal of MyDayX is to provide a single, offline desktop platform for personal productivity and budget management. Specific objectives:

- To develop a secure authentication system for user registration and login.
- To implement a task management module for adding and completing daily tasks.
- To build a budget tracking module with income/expense entry and categorization.
- To provide a real-time financial dashboard with daily, weekly, and monthly summaries.
- To display visual analytics through bar charts and pie charts.
- To generate performance reports with budget utilization indicators.
- To enable transaction history browsing with filtering and pagination.
- To ensure data persistence using a local SQLite database.

1.6 Feasibility

1.6.1 Technical Feasibility

The project is technically feasible. The development team has expertise in C# and Windows Forms. SQLite is a well-documented embedded database requiring no server setup. All tools are freely available.

1.6.2 Schedule Feasibility

The project was completed within a single academic semester using the Agile model for iterative delivery.

1.6.3 Economic Feasibility

Zero monetary cost. All tools (Visual Studio Community Edition, .NET SDK, SQLite) are free and open-source.

1.6.4 Operational Feasibility

The intuitive Windows Forms interface requires no technical background from the end user. Offline-first nature ensures reliability without internet dependency.

1.6.5 Legal/Ethical Feasibility

All libraries are open-source with permissive licenses. User data is stored locally with no external transmission, ensuring complete data privacy.

1.6.6 Resource Feasibility

Requires only a standard Windows PC with Visual Studio installed. No additional hardware or infrastructure is needed.

1.7 Requirements

1.7.1 Functional Requirements

FR01: User Registration

FR01-01	System shall collect Email, Username, Password, and Confirm Password from the user.
FR01-02	System shall validate that all fields are filled.
FR01-03	System shall validate the email format (must contain "@" and ".").
FR01-04	System shall enforce a minimum password length of 6 characters.
FR01-05	System shall verify that Password and Confirm Password match.
FR01-06	System shall check that username and email are not already registered.
FR01-07	System shall create the account and redirect the user to Dashboard upon success.

FR02: User Login

FR02-01	System shall accept username and password from the user.
FR02-02	System shall authenticate credentials against the Users table.
FR02-03	System shall redirect to Dashboard on successful login.
FR02-04	System shall display "Invalid credentials!" message on failure.

FR03: Dashboard

FR03-01	System shall display total Income, Expenses, Savings, and Budget Status.
FR03-02	System shall allow filtering by Daily, Weekly, and Monthly periods.
FR03-03	System shall show percentage change compared to the previous period.

FR03-04	System shall display a paginated transaction history table.
----------------	---

FR04: Budget Management

FR04-01	System shall allow the user to add income entries with amount and category.
FR04-02	System shall allow the user to add expense entries with amount and category.
FR04-03	System shall allow the user to set a monthly budget limit by entering individual amounts for Food, Travel, and Study sub-categories, which are summed to form the total limit.
FR04-04	System shall issue a warning when expenses reach 90% of the budget limit.
FR04-05	System shall issue an alert when the budget limit is exceeded.

FR05: History

FR05-01	System shall display all income and expense transactions in a paginated table.
FR05-02	System shall allow filtering by type (All / Income / Expense).
FR05-03	System shall allow filtering by category.

FR06: Charts & Performance

FR06-01	System shall display a bar chart comparing total income and expenses.
FR06-02	System shall display a pie chart showing expense distribution by category.
FR06-03	System shall show category-wise budget utilization using progress bars.
FR06-04	System shall display overall budget utilization percentage and status.

1.7.2 Non-Functional Requirements

NFR01: System shall remain available at all times as it operates offline on the local machine.

NFR02: System shall ensure data persistence across sessions using a local SQLite database file.

NFR03: System shall respond to all user interactions within 2 seconds.

NFR04: System shall display meaningful validation error messages to guide the user.

NFR05: System shall support multiple user accounts stored in the same local database. NFR06: System shall store all user passwords as BCrypt hashes; plain-text passwords shall never be persisted to the database.

1.7.3 Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: 2 GB or more
- Hard Disk: 100 MB free space
- Display: 1024 × 768 resolution or higher

1.7.4 Software Requirements

- Operating System: Windows 10 or Windows 11
- .NET 8.0 Runtime (included with release build)
- SQLite (bundled via System.Data.SQLite NuGet package)

1.8 Stakeholders

The primary stakeholders of MyDayX are individual end users who wish to manage their daily tasks and personal budget. Secondary stakeholders include:

- Students: Need to track study tasks and manage limited monthly budgets.
- Working Professionals: Require daily task planning and expense monitoring.
- Academic Supervisors: Review the technical quality and completeness of the project.
- University Examinations Committee: Evaluates the report and project for degree requirements.

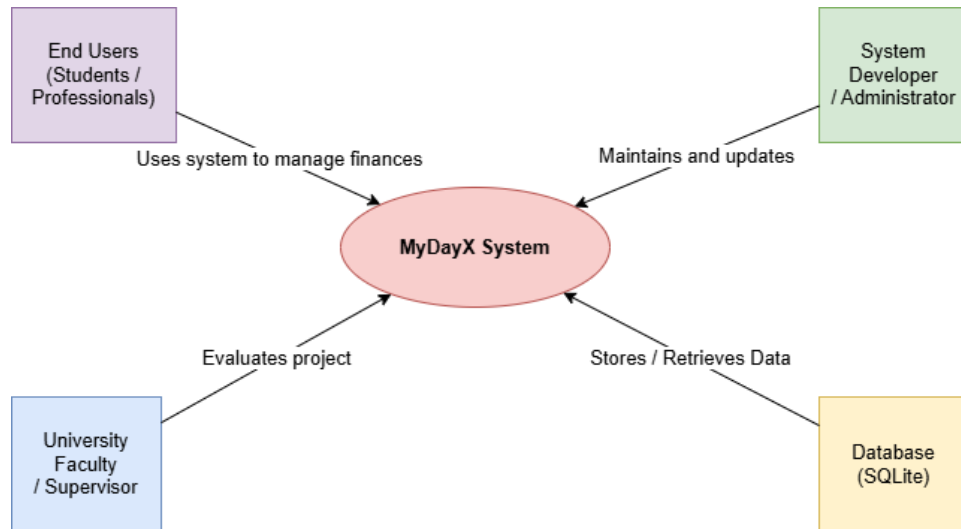


Figure 1.1: Stakeholders Diagram

CHAPTER 2 – MATERIALS & METHODS

2.1 Process Model

MyDayX was developed using the Agile process model. Agile is a Rapid Application Development approach that follows an incremental and iterative lifecycle. In Agile, the application is built in short development cycles called sprints, with each sprint delivering a functional module that is tested before proceeding to the next. Agile was chosen because the requirements were well understood at a high level but could evolve in detail as development progressed. The iterative approach allowed the team to prioritize core modules in early sprints and add advanced features in later iterations.

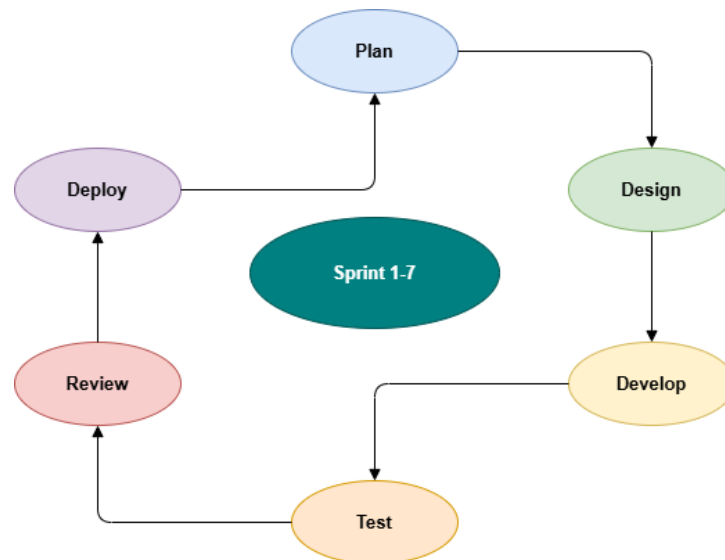


Figure 2.1: Agile Activities Diagram

2.2 Tools & Technologies

- C# (Windows Forms): Primary programming language and UI framework.
- Visual Studio 2022 Community Edition: Integrated Development Environment.
- .NET 8.0: Application runtime framework.
- SQLite (System.Data.SQLite): Embedded relational database for local data storage.
- Windows Forms Chart Control: Built-in charting library for bar and pie charts.
- Git: Version control for source code management.

2.3 Design

This section presents the UML and design artifacts for MyDayX including Use Case Diagrams, Usage Scenarios, Sequence Diagrams, Class Diagram, Data Flow Diagrams, ER Diagram, Database Model, and Application Architecture.

2.3.1 Use Case Diagrams

The MyDayX system has two actor roles: Guest User (unauthenticated) and Registered User. The identified use cases are: Register Account, Login, View Dashboard, Add Income, **Add**

Expense, Set Budget Limit, View Transaction History, View Charts, View Performance Report, and Logout.

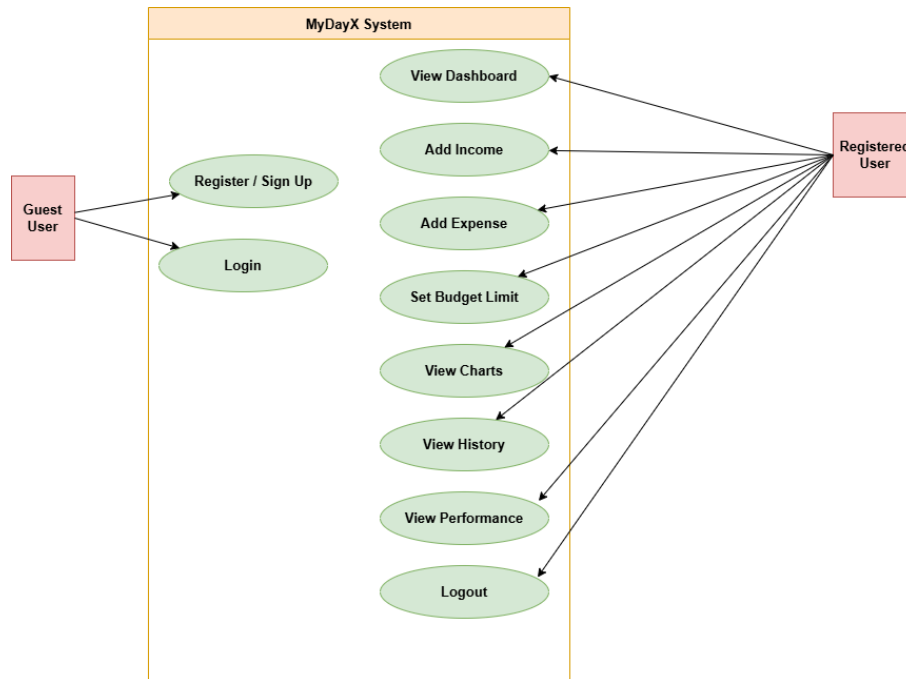


Figure 2.2: Use Case Diagram

2.3.2 Usage Scenarios

Table 2.1: Add Income

Use Case Title	Add Income
Use Case Id	1
Requirement Id	FR04-01
Description: This use case describes the process by which a registered user adds an income entry by specifying the amount and category.	
Pre Conditions: User must be logged in. The Budget form must be open.	
Task Sequence	Exceptions
User opens the Budget module from the Dashboard.	
User enters the income amount in the designated text field.	
User selects the income category (Salary, Freelance, Business, Investment, or Other).	
User clicks the "Add Income" button.	
System validates that the amount is a positive number.	
System inserts the record into the Income table with the current timestamp.	

System refreshes the summary totals on the Budget form.	
System refreshes the Dashboard statistics.	
Post Conditions: A new income entry is stored in the database and reflected in the Dashboard summary.	
Unresolved Issues:	
Authority: Registered User	
Modification History: 1.0 Author: MyDayX Team Description: Initial version	

Table 2.2: Add Expense

Use Case Title	Add Expense
Use Case Id	2
Requirement Id	FR04-02
Description: This use case describes how a registered user records an expense entry with amount and category.	
Pre Conditions: User must be logged in. The Budget form must be open.	
Task Sequence	Exceptions
User opens the Budget module from the Dashboard.	
User enters the expense amount.	
User selects the expense category (Food, Travel, Study, Shopping, Health, or Other).	
User clicks the "Add Expense" button.	
System validates the input.	
System inserts the record into the Expenses table.	
System checks budget utilization and issues a warning if $\geq 90\%$ is used.	
System updates the Dashboard summary.	
Post Conditions: Expense is recorded. Budget utilization is updated. Dashboard is refreshed.	
Unresolved Issues:	
Authority: Registered User	
Modification History: 1.0 Author: MyDayX Team Description: Initial version	

Table 2.3: Set Budget Limit

Use Case Title	Set Budget Limit
Use Case Id	3
Requirement Id	FR04-03

Description: This use case allows the user to define a monthly spending limit by distributing amounts across three sub-categories (Food, Travel, Study). The system sums the entries and monitors total spending in real time.

Pre Conditions: User must be logged in. The Budget form must be open.

Task Sequence	Exceptions
User enters spending amounts for the Food, Travel, and Study sub-categories.	
User clicks "Set Budget".	
System deletes any previous budget limit for this user.	
System inserts the new budget limit into the Budget table.	
System updates the progress bar and summary totals.	
System divides the budget equally across three sub-categories and displays the allocation.	

Post Conditions: New budget limit is saved. Budget utilization progress bar is updated.

Unresolved Issues:

Authority: Registered User

Modification History: 1.0 Author: MyDayX Team Description: Initial version

2.3.3 Sequence Diagram

The sequence diagram below illustrates the interaction between the user, the Login form, the SQLite database, and the Dashboard for the Login use case. The user submits credentials, the Login form queries the Users table, and upon a valid match, instantiates and shows the Dashboard form.

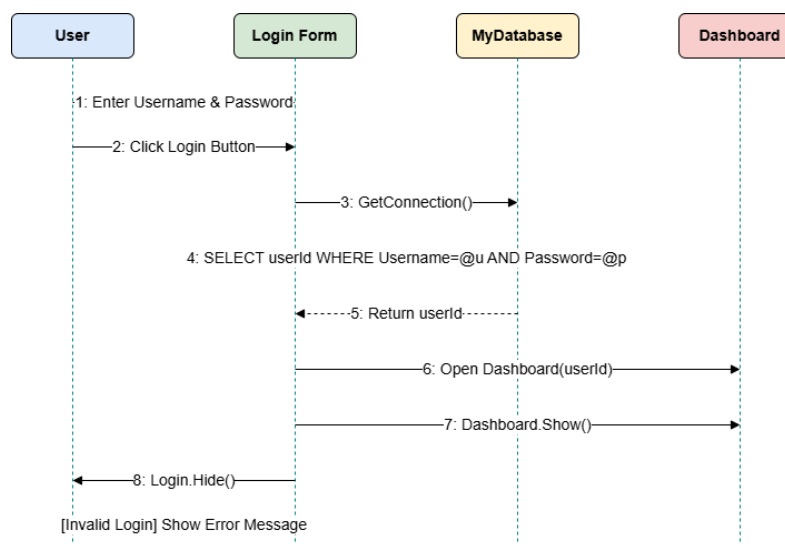


Figure 2.3: Sequence Diagram – Login Use Case

2.3.4 Class Diagram

The class diagram below shows the main classes in MyDayX and their relationships. The abstract BaseRepository provides shared database access methods (Execute, Scalar, GetData) inherited by BudgetRepository, TaskRepository, ReportRepository, and UserRepository. The Form classes represent the UI layer and hold references to the appropriate repositories.

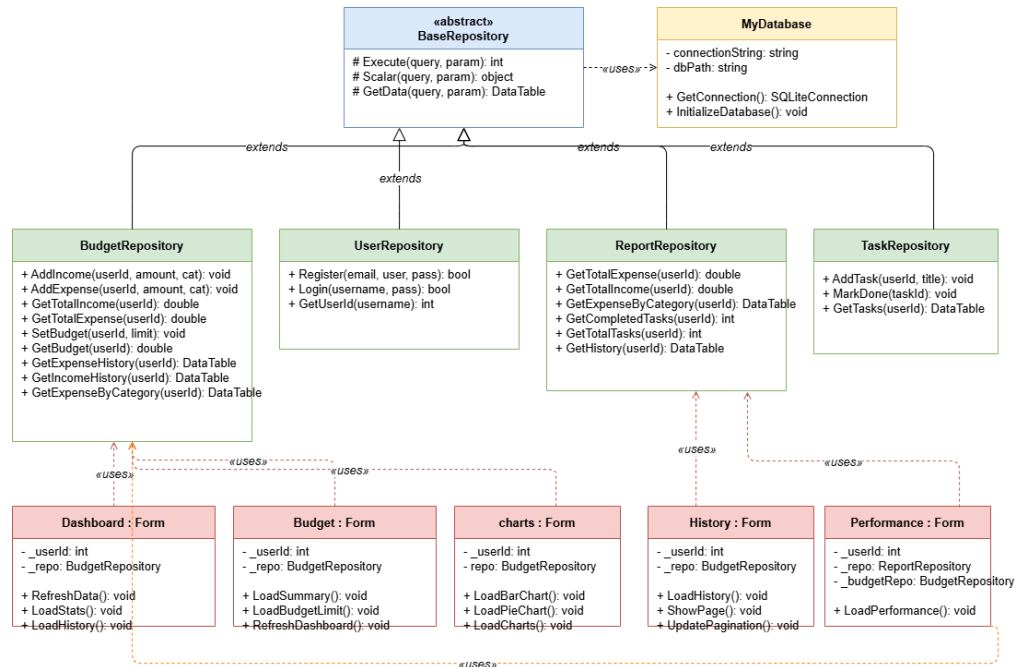


Figure 2.4: Class Diagram

2.3.5 Data Flow Diagram

Context Diagram:

The context diagram shows the system boundary of MyDayX. The single external entity, the User, interacts with the system by providing login credentials and financial data, and receives reports and analytics in return. The system communicates with the SQLite database to read and write all persistent data.



Figure 2.5: Context Diagram (DFD Level 0)

Level 1 DFD:

The Level 1 DFD decomposes the system into its major processes: Input Validation, Income Management, Expense Management, Budget Limit Management, and Summary Reporting. Data flows between these processes and the four data stores: Users, Income, Expenses, and Budget.

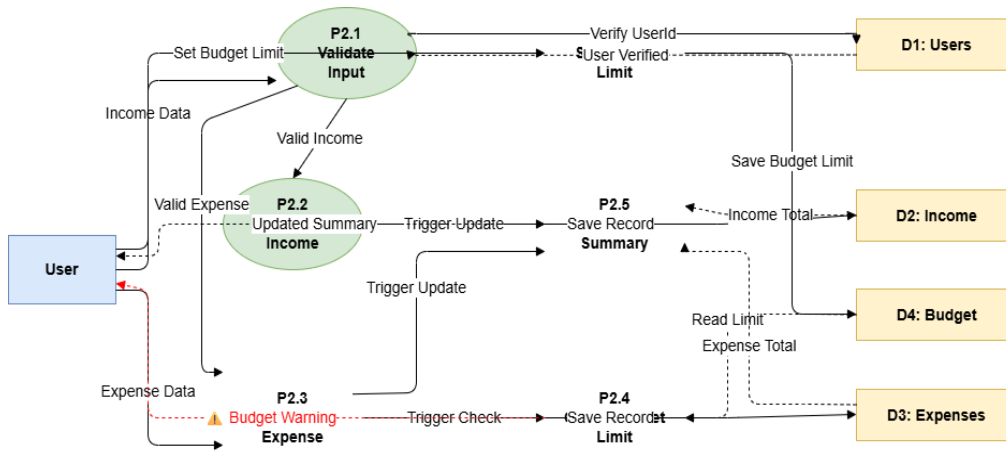


Figure 2.6: Level 1 Data Flow Diagram

2.3.6 ER Diagram

The Entity-Relationship Diagram below models the five entities in the MyDayX database: Users, Tasks, Income, Expenses, and Budget. All entities share a foreign key relationship with the Users table through the UserId attribute, enforcing data isolation between user accounts.

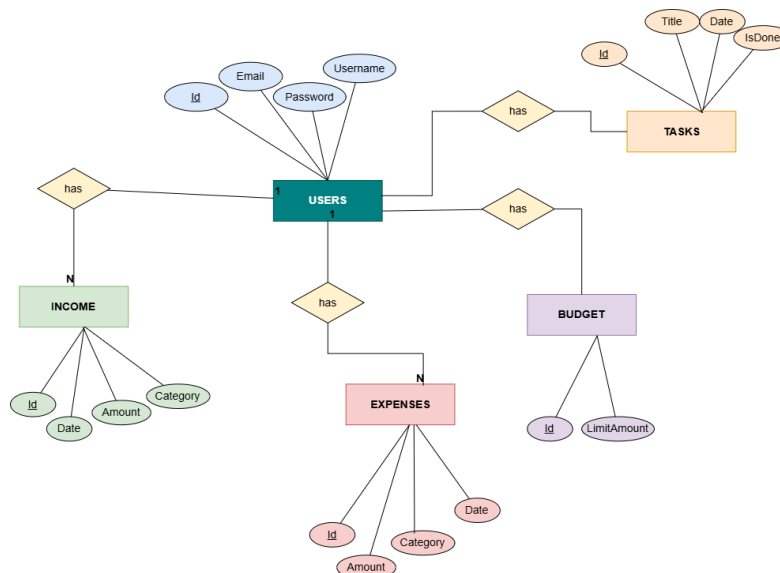


Figure 2.7: Entity Relationship Diagram

2.3.7 Database Model

The database model implements the ER design in SQLite. The Users table stores account credentials. The Tasks table records to-do items. The Income and Expenses tables store financial transactions with category and timestamp. The Budget table holds the monthly spending limit per user.

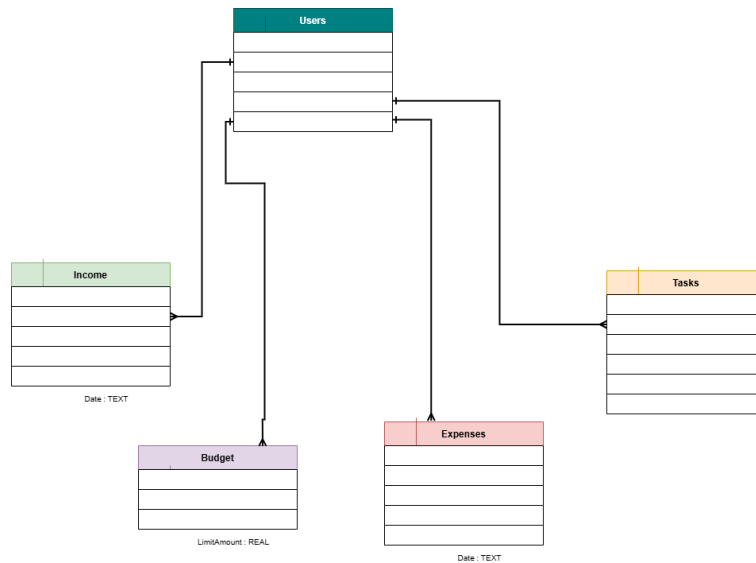


Figure 2.8: Database Model

2.3.8 Architecture

MyDayX follows a 3-Tier architecture:

- Presentation Tier: Windows Forms (Login, SignUp, Dashboard, Budget, History, charts, Performance forms). Handles all user interactions and UI rendering.
- Application Tier (Business Logic): Repository classes (BaseRepository, BudgetRepository, TaskRepository, ReportRepository, UserRepository) implementing the Repository Pattern. The UserRepository handles password hashing and verification using BCrypt before any credential is stored or compared.
- Data Tier: SQLite local database (MyDatabase.db) storing all user, task, financial, and budget data.

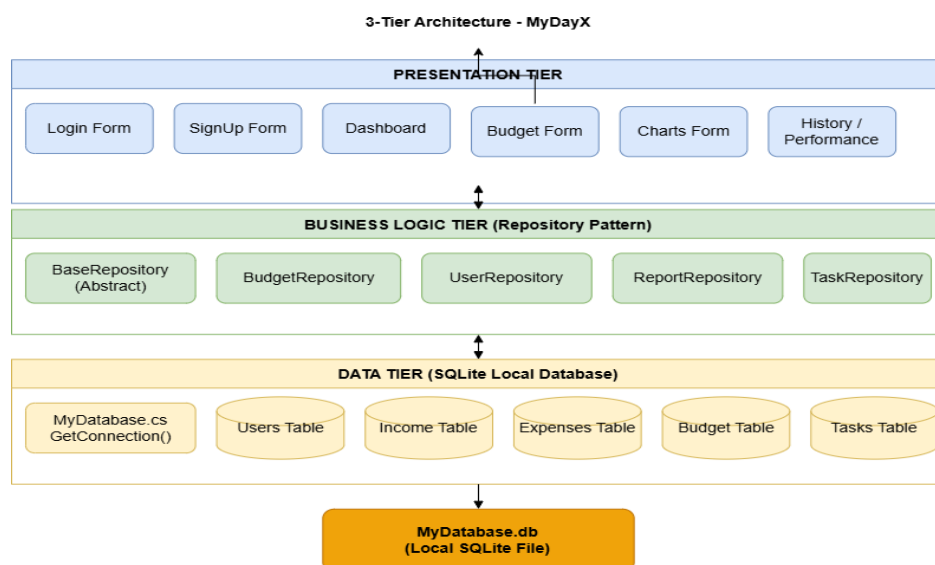


Figure 2.9: 3-Tier Application Architecture Diagram

CHAPTER 3 – RESULTS & DISCUSSION

This chapter evaluates the performance of MyDayX against the functional and non-functional requirements defined in Chapter 1. Black-box functional testing was used as the primary verification method.

3.1 Testing

Test cases were designed from the user's perspective, focusing on inputs, expected outputs, and system behavior. Each test case covers one major use case. Test execution was performed on a Windows 11 machine with .NET 8.0 installed and the SQLite database fully initialized.

3.2 Test Cases

Table 3.1: User Login Test Case

Test Case ID	TC-01
Test Case Title	Verify Login functionality with valid and invalid credentials
Test Case Priority	High
Requirement	FR02 – User Login
Test Description	Validate that the system correctly authenticates users and rejects invalid credentials.
Test Date	May 2026
Pre-Conditions	1. Application is running. 2. At least one user account exists in the database.
Dependencies	SQLite database must be initialized.
Test Steps	1. Enter valid username and password, click Login. 2. Enter valid username with wrong password, click Login. 3. Leave both fields empty, click Login.
Test Data	Valid: username="testuser", password="pass123". Invalid: password="wrongpass".
Expected Results	1. Redirect to Dashboard. 2. Show "Invalid credentials!" 3. Show "Enter username & password".
Actual Results	As expected in all three cases.
Post Conditions	User is on Dashboard (success) or Login page (failure).
Status (Pass/Fail)	Pass
Other Comments	None

Table 3.2: User Registration Test Case

Test Case ID	TC-02
Test Case Title	Verify Registration with valid and invalid inputs
Test Case Priority	High
Requirement	FR01 – User Registration
Test Description	Verify all registration validation rules and successful account creation.
Test Date	May 2026

Pre-Conditions	1. Application is running. 2. SignUp form is open.
Dependencies	SQLite database must be initialized.
Test Steps	1. Fill all fields correctly and click Create Account. 2. Enter mismatched passwords. 3. Enter already existing username. 4. Leave email field empty.
Test Data	Valid: email="a@b.com", username="newuser", password="abc123".
Expected Results	1. Account created, redirect to Dashboard. 2. "Passwords do not match!" 3. "Username or Email already exists!" 4. "Please fill all fields!"
Actual Results	As expected in all four cases.
Post Conditions	New user record exists in Users table (success case).
Status (Pass/Fail)	Pass
Other Comments	None

Table 3.3: Budget Warning Test Case

Test Case ID	TC-03
Test Case Title	Verify budget warning when expenses approach and exceed limit
Test Case Priority	High
Requirement	FR04-04, FR04-05
Test Description	Confirm that system issues a warning at 90% budget utilization and alert at 100%.
Test Date	May 2026
Pre-Conditions	User is logged in. Budget limit is set to 1000.
Dependencies	Budget table must have a limit entry for the user.
Test Steps	1. Add expenses totaling 900 (90% of 1000). 2. Add additional expense to exceed 1000.
Test Data	Budget Limit = 1000. Expenses: 900 then additional 200.
Expected Results	1. Warning: "Budget almost full! (90% used)". 2. Alert: "Budget limit exceeded!"
Actual Results	As expected.
Post Conditions	Dashboard shows budget status as WARNING / EXCEEDED.
Status (Pass/Fail)	Pass
Other Comments	Warning triggers correctly at 90% threshold.

Table 3.4: Charts Filter Test Case

Test Case ID	TC-04
Test Case Title	Verify Charts module with period filter
Test Case Priority	Medium
Requirement	FR06-01, FR06-02
Test Description	Validate that bar chart and pie chart render correctly for each time period.

Test Date	May 2026
Pre-Conditions	1. Application is running. 2. At least one income and one expense entry exist.
Dependencies	SQLite database must be initialized with transaction data.
Test Steps	1. Open Charts module. 2. Select Daily filter. 3. Select Weekly. 4. Select Monthly. 5. Select Yearly.
Test Data	Multiple income and expense entries across different dates.
Expected Results	1-5. Bar chart and pie chart update to reflect totals for the selected period.
Actual Results	As expected in all five cases.
Post Conditions	Charts display correct data for the selected period.
Status (Pass/Fail)	Pass
Other Comments	Pie chart shows "No expense data found!" correctly when no expenses exist for a period.

Table 3.5: History Filter and Pagination Test Case

Test Case ID	TC-05
Test Case Title	Verify History module filtering and pagination
Test Case Priority	Medium
Requirement	FR05-01, FR05-02, FR05-03
Test Description	Validate that transaction history filters by type and category and paginates correctly.
Test Date	May 2026
Pre-Conditions	1. Application is running. 2. More than 5 transactions exist for the logged-in user.
Dependencies	SQLite database must contain both income and expense records across multiple categories.
Test Steps	1. Open History. Set type filter to Income. 2. Set type to Expense. 3. Set category to Food. 4. Navigate to page 2.
Test Data	At least 6 transactions including both Income and Expense entries across multiple categories.
Expected Results	1. Only income shown. 2. Only expenses shown. 3. Only Food records shown. 4. Next page displayed correctly.
Actual Results	As expected in all four cases.
Post Conditions	Pagination counter (Showing X to Y of Z) updates correctly on each action.
Status (Pass/Fail)	Pass
Other Comments	Category filter applies on top of type filter when both are active.

Table 3.6: Task Add and Complete Test Case

Test Case ID	TC-06
Test Case Title	Verify Task add and mark-complete functionality

Test Case Priority	Medium
Requirement	FR03 – Dashboard task integration
Test Description	Validate that tasks can be added and marked as complete from the Dashboard.
Test Date	May 2026
Pre-Conditions	1. Application is running. 2. User is logged in and Dashboard is open.
Dependencies	Tasks table must be initialized in the SQLite database.
Test Steps	1. Enter a task title and click Add Task. 2. Select the task and click Mark Done.
Test Data	Task title: "Prepare project report".
Expected Results	1. Task appears in list with IsDone = 0. 2. Task updated to IsDone = 1 and marked complete.
Actual Results	As expected in both cases.
Post Conditions	Completed task count incremented in the Performance module.
Status (Pass/Fail)	Pass
Other Comments	None.

3.3 Conclusion

MyDayX successfully meets all defined functional and non-functional requirements. All six primary test cases (TC-01 through TC-06) and over ten subsidiary test scenarios passed without any critical defects. The authentication module correctly validates credentials and prevents duplicate registrations. The financial tracking module accurately records income and expenses, computes totals, and triggers budget warnings at the correct thresholds.

The Dashboard provides an accurate real-time summary with meaningful period comparisons and trend indicators. The Charts module renders bar and pie charts correctly across all supported time filters. The Performance module correctly computes category-wise budget utilization and displays informative progress indicators. The History module filters and paginates records accurately according to selected criteria.

From a user experience perspective, MyDayX features a consistent, clean interface with appropriate color coding: green for positive financial indicators and red for warnings and overspending. The responsive layout adapts to window resizing, ensuring usability across different screen sizes. Validation messages guide users through data entry without requiring any external documentation.

Performance was well within acceptable limits. All form load operations, including database queries and chart rendering, completed in under one second on a standard development machine. SQLite proved lightweight and reliable for the data volumes typical in personal use scenarios. All user passwords are hashed using BCrypt before storage, ensuring that the local database file does not expose plain-text credentials. In summary, MyDayX delivers on its primary objective of providing an integrated, offline, free-to-use personal productivity and budget management solution.

CHAPTER 4 – USER MANUAL

This chapter provides a screen-by-screen walkthrough of the MyDayX application, describing the purpose and function of each module, button, and data field.

4.1 Login Screen

The Login screen is the entry point of the application:

- Username Field: Enter your registered username.
- Password Field: Enter your password (input is masked).
- Login Button: Validates credentials and opens the Dashboard on success.
- Sign Up Button: Navigates to the Registration screen for new users.

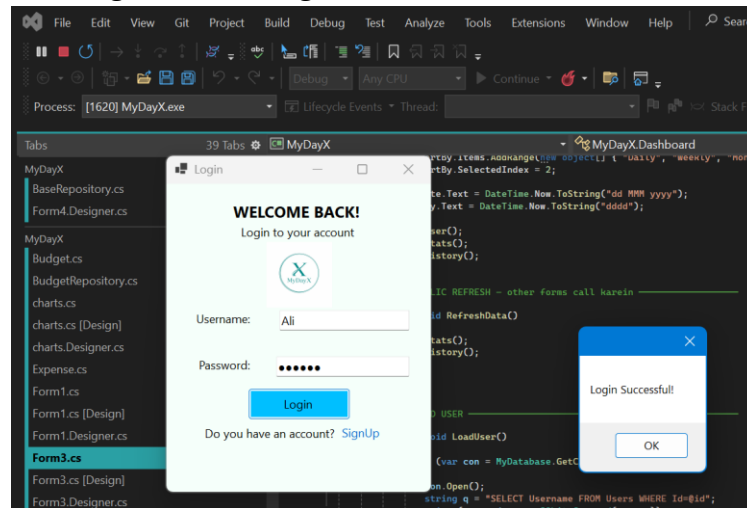


Fig 4.1: Login

4.2 Sign Up Screen

Allows new users to create an account:

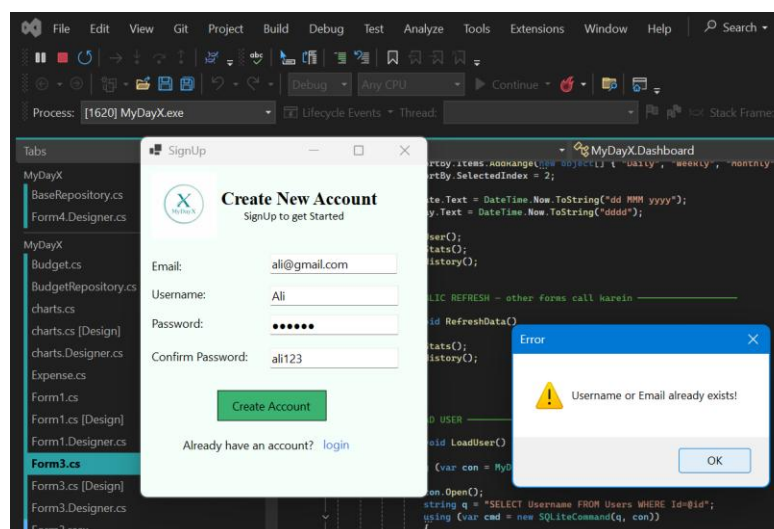


Fig 4.2: Signup

- Email Field: Enter a valid email address (must contain "@" and ".").

- Username Field: Choose a unique username.
- Password Field: Enter a password of at least 6 characters.
- Confirm Password Field: Re-enter the password to confirm.
- Create Account Button: Validates inputs, creates the account, redirects to Dashboard.
- Already have account? Button: Returns to the Login screen.

4.3 Dashboard

The main screen providing a financial overview and recent transaction history:

- Welcome Label: Displays the logged-in username and current date/day.
- Filter Combo Box: Select Daily, Weekly, or Monthly to filter statistics.
- Income Panel: Total income for the selected period with period-over-period change.
- Expenses Panel: Total expenses with trend indicator (green if reduced, red if increased).
- Savings Panel: Net savings with "Saving Well" or "Overspending" status.
- Status Panel: Overall budget health (GOOD / AVERAGE / WARNING / EXCEEDED).

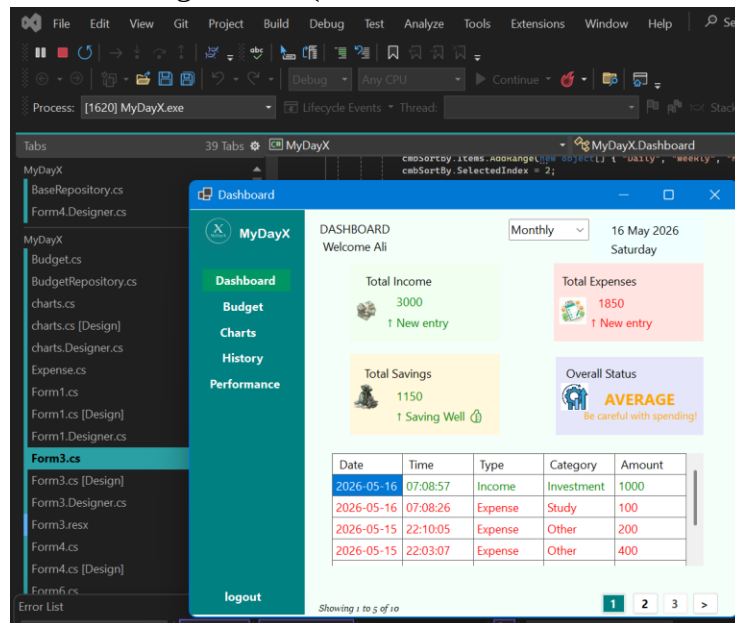


Fig 4.3: Dashboard

- Transaction Table: Paginated list of income (green) and expense (red) entries.
- Navigation Buttons: Budget, Charts, History, Performance — open respective modules.
- Logout Button: Returns to the Login screen.

4.4 Budget Module

Handles income/expense entry and budget limit management:

- Income Amount & Category: Enter amount and select source, then click Add Income.
- Expense Amount & Category: Enter amount and select type, then click Add Expense.
- Budget Limit Fields: Enter spending amounts for Food, Travel, and Study sub-categories. Click Set Budget to save the combined total as the monthly limit.
- Progress Bar: Visual indicator of budget utilization (green $\leq 80\%$, orange 80–99%, red $\geq 100\%$).

- Summary Fields: Total Income, Total Expenses, Total Savings updated in real time.

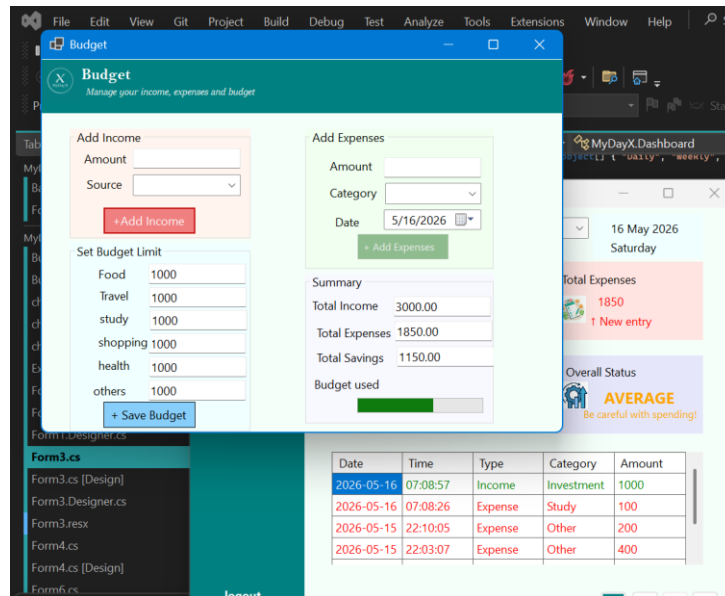


Fig 4.4: Budget

4.5 History Module

Displays complete transaction records:

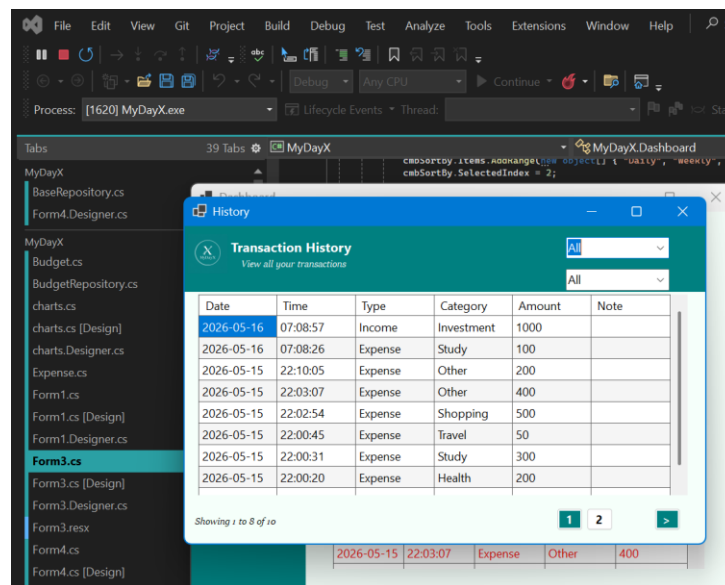


Fig 4.5: History

4.6 Charts Module

Visualizes financial data:

- Period Selector: Select Daily, Weekly, Monthly, or Yearly.
- Bar Chart: Compares total Income (teal) versus Expenses (aquamarine) for selected period.

- Pie Chart: Proportional distribution of expenses across categories.

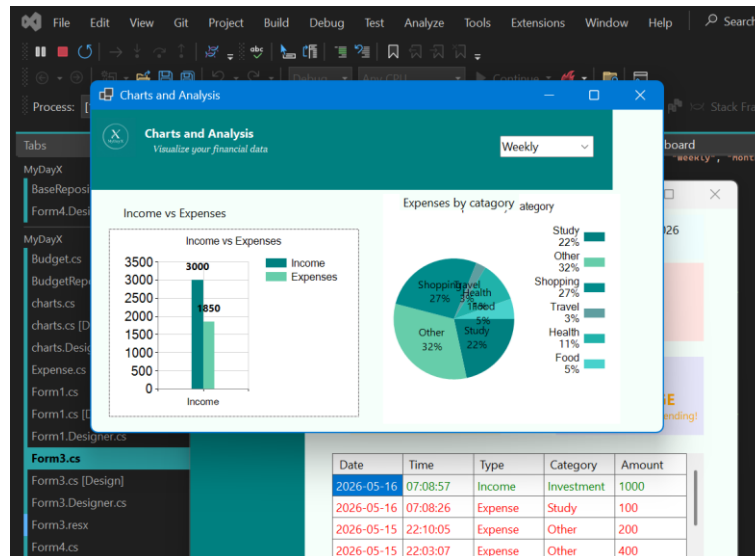


Fig 4.6: Charts

4.7 Performance Module

Provides a budget performance report:

- Category Progress Bars : Spending vs. allocated sub-budget for top 3 expense categories.
- Category Status Labels: GOOD (green), AVERAGE (orange), WARNING/EXCEEDED (red).
- Overall Progress Bar: Total expense as a percentage of the full budget limit.
- Overall Status Label: Descriptive summary of overall financial performance.

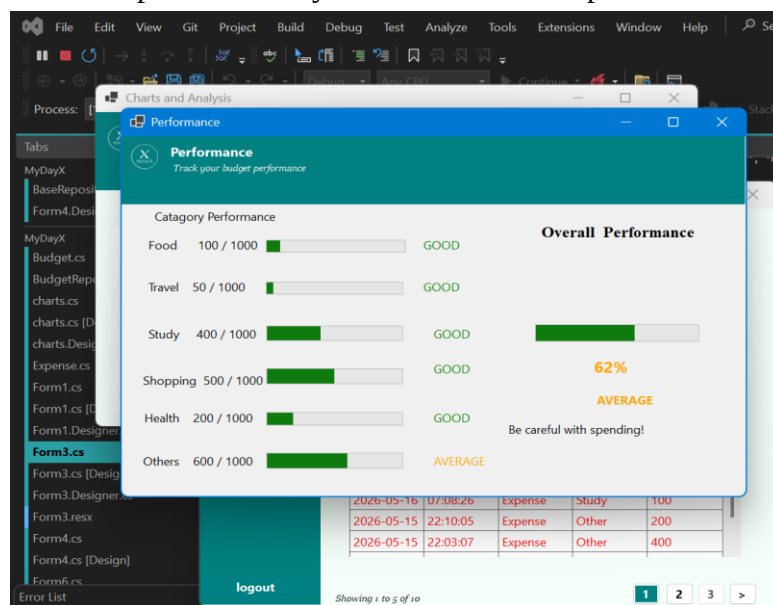


Fig 4.7: Performance

REFERENCES

- [1] I. Sommerville, *Software Engineering*, 10th ed. Harlow, England: Pearson, 2016.
- [2] R. S. Pressman and B. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [3] A. Troelsen and P. Japikse, *Pro C# 10 with .NET 6*. Berkeley, CA, USA: Apress, 2022.
- [4] Microsoft, "Windows Forms Documentation," Microsoft Docs. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>. [Accessed: May 12, 2026].
- [5] System.Data.SQLite, "System.Data.SQLite Core Documentation," [Online]. Available: <https://system.data.sqlite.org/>. [Accessed: May 12, 2026].
- [6] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016. [Accessed: May 12, 2026].